Cryptographic perfect hash functions

A theoretical analysis on space efficiency and entropy

Alexander Towell
atowell@siue.edu

Abstract

We analyze a theoretical *perfect hash function* that has three desirable properties: (1) it is a cryptographic hash function; (2) its *in-place* encoding obtains the theoretical lower-bound for the expected space complexity; and (3) its *in-place* encoding is a random bit string with maximum entropy.

Keywords: perfect hash function, random oracle, space complexity, maximum entropy, cryptographic hash function

Contents

1 Perfect hash functions **1**

2 A theoretical *cryptographic* perfect hash function **4**

2.1 Entropy 7

Appendices **8**

A k-perfect hash function **8**

B Probability mass of random bit length **10**

Glossary **13**

1 Perfect hash functions

A *set* is given by the following definition.

Definition 1. *A set is an unordered collection of distinct elements from a universe of elements.*

We are exclusively interested in the countable sets. A countable set is a *finite set* or a *countably infinite set*. A *finite set* has a finite number of elements. For example,

$$\mathbb{S} = \{1, 3, 5\}$$

is a finite set with three elements. A *countably infinite set* can be put in one-to-one correspondence with the set of *natural numbers* given by

$$\mathbb{N} = \{1, 2, 3, 4, 5, \ldots\}\,. \tag{1}$$

The cardinality of a set $\mathbb{S}$ is a measure of the number of elements in the set, denoted by

$$|\mathbb{S}| \,. \tag{2}$$

The cardinality of a *finite set* is a natural number that counts the number of elements in the set, e.g.,

$$\big|\{1,3,5\}\big| = 3\,.$$

We consider sets in which the universe of elements is given by the *countably infinite* set of all bit strings.

Definition 2. *The countably infinite set of all bit strings is denoted by*

$$\mathbb{B} = \big\{b\colon b \in \{0,1\}^*\big\}\,, \tag{3}$$

where $*$ denotes all non-negative integers.

A *finite* subset of $\mathbb{B}$ is given by the following definition.

Definition 3. *The* finite set *of all bit strings of length n is denoted by*

$$\mathbb{B}_n = \big\{b\colon b \in \{0,1\}^n\big\} \tag{4}$$

with a cardinality given by

$$|\mathbb{B}_n| = 2^n\,. \tag{5}$$

The bit length of an object x is denoted by

$$\mathtt{BL}(x)\,, \tag{6}$$

e.g., the bit length of any $x \in \mathbb{B}_n$ is $\mathtt{BL}(x) = n$.

A (convenient) one-to-one correspondence between $\mathbb{B}$ and $\mathbb{N}$ is given by the following definition.

Definition 4. *Let the set of bit strings $\mathbb{B}$ and the set of natural numbers $\mathbb{N}$ have the bijection given by*

$$(b_1\, b_2\, \cdots\, b_m) \longleftrightarrow 2^m + \sum_{j=1}^{m} 2^{m-j} b_j\,. \tag{7}$$

We denote the mapping of a bit string (or natural number) x by x'.

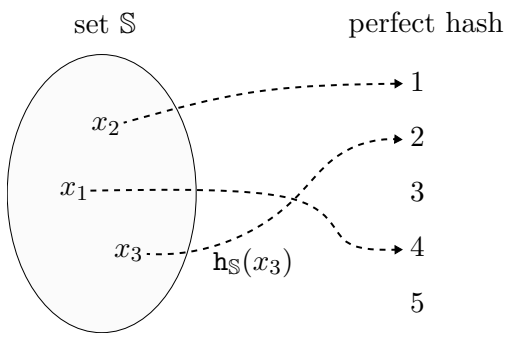
An important observation of this mapping is that a natural number n maps to a bit string n' of length $\mathtt{BL}(n') = \lfloor \log_2 n \rfloor$.

Example 1 Consider the number 100 (base 10). To determine the bit string $100'$, we perform the following steps:

1. The length of $100'$ is $\lfloor \log_2 100 \rfloor = 6$.
2. Subtract $2^6 = 64$ from 100 which results in 36.
3. 36_{10} in binary is 100100_2.
4. $100' = (1\,0\,0\,1\,0\,0)$.

A *hash function* is related to countable sets $\mathbb{B}$ and $\mathbb{B}_n$ and is given by the following definition.

Figure 1: Perfect hash function



Definition 5. *A* hash function $\mathtt{h}\colon \mathbb{B} \mapsto \mathbb{B}_n$ *is a function such that all bit strings of arbitrary-length are mapped (hashed) to bit strings of fixed-length* n*.*

For a given $x \in \mathbb{B}$, $\mathtt{h}(x)$ is denoted the *hash* of x.

We are particularly interested in *perfect* hash functions as given by the following definition.

Definition 6 (Perfect hash function)**.** *A* perfect hash function *of the finite set* $\mathbb{S} \subset \mathbb{B}$*, denoted by*

$$\mathtt{h}_{\mathbb{S}}\colon \mathbb{B} \mapsto \mathbb{Z}^+\,, \tag{8}$$

is an injective function over $\mathbb{S}$ *such that the mapping from* $\mathbb{S}$ *to* $\mathbb{Z}^+$ *is* one-to-one*. That is, no collisions among elements of* $\mathbb{S}$ *occur,*

$$\mathtt{h}_{\mathbb{S}}(x) \neq \mathtt{h}_{\mathbb{S}}(y) \tag{9}$$

for all $x, y \in \mathbb{S}$ *where* $x \neq y$*.*

The *load factor* is given by the following definition.

Definition 7. *The proportion of hashes mapped to by a perfect hash function is denoted the load factor. The load factor of* $\mathtt{h}_{\mathbb{S}}$ *is a rational number given by*

$$r = \frac{m}{N}\,, \tag{10}$$

where $m = |\mathbb{S}|$ *and* N *is the maximum possible hash of* $\mathtt{h}_{\mathbb{S}}$*.*

Example 2 Consider the set $\mathbb{S} = \{x_1, x_2, x_3\}$. If we wish to construct a perfect hash function of the set with a load factor $\frac{3}{5}$, then the *maximum* hash must be 5. See fig. 1 for an illustration.

Definition 8. *A* minimal *perfect hash has a load factor* 1*.*

A *minimal* perfect hash function is a special case of an *approximate map*[3] with a false positive rate 1 and false negative rate 0 where the *keys* are the set of bit strings $\mathbb{S}$ and the *values* are the set given by $\{1, 2, \ldots, m\}$, $m = |\mathbb{S}|$.

2 A theoretical *cryptographic* perfect hash function

A random oracle as given by the following definition.

Definition 9. *A random oracle is a hypothetical hash function* $\mathtt{h}^*\colon \mathbb{B} \mapsto \mathbb{B}_n$ *whose output is uniformly distributed over the elements of* $\mathbb{B}_n$*.*

The theoretical analysis of the perfect hash function makes the following assumptions.

Assumption 1. *We have a hash function* $\mathtt{h}$ *that is a good approximation of a random oracle with a space complexity* $\mathcal{O}(1)$*.*

Assumption 2. *The set* $\mathbb{S}$ *consists of* m *elements, where each element is drawn uniformly at random without replacement from* $\mathbb{B}$*.*

Algorithm 1: Perfect hash function generator

```
   in  : 𝕊 is a set and and r is a load factor.
   out : A perfect hash function of 𝕊 with a load factor r.
1  function make_perfect_hash(𝕊, r)
2      m ← |𝕊|;
3      N ← ⌈m/r⌉;
4      for n ← 1 to ∞ do
           // We give a name to the hash function that is coded by n′. Note that
              ++ is the concatenation operator.
5          ĥ(x) ≡ h(x++n′) mod N;
6          if no collisions occur for hash function ĥ on 𝕊 then
               // This tuple is sufficient to code the perfect hash function for 𝕊
                  that has the minimum bit length.
7              return (n′, N);
8          end
9      end
```

We search *all* possible hash functions which are a function of $\mathtt{h}$, an approximate random oracle, and choose the *perfect hash function* on a specified set that has the *smallest* bit length. We describe the algorithm for performing this exhaustive search in algorithm 1.

We consider a family of perfect hash functions for a set $\mathbb{S}$ which are given by the output of a hash function $\mathtt{h}$ that approximates a *random oracle* applied to the input $x \in \mathbb{S}$ concatenated with a bit string $b \in \mathbb{B}$. We describe the generative algorithm for the perfect hash function in algorithm 1. Note that in algorithm 1, the concatenation of two bit strings x and y is denoted by $x{+}{+}y$.

The perfect hash function generated by algorithm 1 has the statistical property that the output is uniformly distributed as given by the following theorem.

Theorem 1. *The perfect hash function* $\mathtt{h}_{\mathbb{S}}$ *approximates a* random oracle *that perfectly hashes* $\mathbb{S}$*.*

Proof. First, in algorithm 1 on Line 5, b_n is a bit string such that each $x \in \mathbb{S}$ concatenated with b_n hashes to a unique integer in $\{1, \ldots, N\}$ by the hash function $\mathtt{h}$, thus the hash function found perfectly hashes the elements of $\mathbb{S}$.

Second, by definition 9, $\mathtt{h}$ approximates a random oracle whose output is uniformly distributed over the elements of $\mathbb{B}_n$. Viewing the elements of $\mathbb{B}_n$ as integers, where $|\mathbb{B}_n| = 2^n$, $\mathtt{h}$ is uniformly distributed over $\{0, 1, \ldots, 2^n - 1\}$.

If $N = k2^n$ for some integer k, the remainder of the output of $\mathtt{h}$ when dividing by N, given by the modulo operator, and adding 1 is uniformly distributed over $\{1, \ldots, N\}$ since each $j \in \{1, \ldots, N\}$ has an equal number of hashes assigned to it by $\mathtt{h}$. If $2^n \gg N$ and $N \neq k2^n$ for some integer k, then it is approximately uniformly distributed and converges to the uniform distribution as $n \to \infty$. □

The probability that no collisions occur for a particular bit string in algorithm 1 is given by the following theorem.

Theorem 2. *The probability that a bit string $b \in \mathbb{B}$ results in a perfect hash function $\mathtt{h}_{\mathbb{S}}$ in algorithm 1 is given by*

$$p(m, N) = \frac{N!}{N^m (N-m)!}\,, \tag{11}$$

where $m = |\mathbb{S}|$ and N is the maximum hash of $\mathtt{h}_{\mathbb{S}}$.

Proof. Suppose we have a set $\mathbb{S} = \{x_1, \ldots, x_m\}$. First, we consider a singular set consisting of x_1. The probability that no collision occurs is 1 since there are no other members to collide with. Second, we add x_2 to the set. The probability that no collision occurs is given by the probability

$$\Pr\left[\mathtt{h}^*(x_2 {+\!\!+} b) \neq \mathtt{h}^*(x_1 {+\!\!+} b)\right] = \frac{N-1}{N}\,. \tag{a}$$

This probability follows from the fact that out of the N hashes to choose from, $N-1$ result in no collision. Extending this logic, the probability that no collision occurs for all m elements is given by

$$p(m, N) = \prod_{i=0}^{m-1} \frac{N-i}{N} \tag{b}$$

$$= \frac{N!}{N^m (N-m)!}\,. \tag{c}$$

□

Equation (19) may be reparameterized with respect to the load factor. By eq. (10), $r = m/N$. Solving this equation with respect to N yields the solution

$$N = \frac{m}{r}\,, \tag{12}$$

and thus plugging in this value of N yields the result

$$p(m, r) = \frac{\left(\frac{m}{r}\right)!}{\left(\frac{m}{r}\right)^m \left(\frac{m}{r} - m\right)!}\,. \tag{13}$$

The expected *in-place* coding size is given by the following theorem.

Theorem 3. *The expected coding size is given approximately by*

$$\log_2 \mathrm{e} - \left(\frac{1}{r} - 1\right) \log_2\left(\frac{1}{1-r}\right) \text{bits/element}\,. \tag{14}$$

Proof.

$$\frac{n}{r}(1-r)\log_2(1-r) - \log_2 n - (u-n)\log_2(1-n/u) \tag{a}$$

$$\left(\frac{1}{r}-1\right)\log_2(1-r)-\frac{1}{n}\log_2 n-\frac{u-n}{n}\log_2\left(1-\frac{n}{u}\right) \tag{b}$$

The space required for the perfect hash function found by algorithm 1 is of the order of the length n of the bit string b_n in the returned tuple. Therefore, for space efficiency, the algorithm exhaustively searches for a bit string in the order of increasing size n.

We are interested in the first case when no collision occurs, which is a geometric distribution with probability of success $p(m,r)$ as given by the discrete random variable

$$\mathrm{Q} \sim \mathrm{GEO}\big(p(m,r)\big)\,, \tag{c}$$

where $p(m,r)$ is given by eq. (13). The expected number of trials for the geometric distribution is given by

$$\mathbb{E}\left[\mathrm{Q}\right]=\frac{1}{p(m,r)}=\frac{\left(\frac{m}{r}\right)^m\left(\frac{m}{r}-m\right)!}{\left(\frac{m}{r}\right)!}\,. \tag{d}$$

By definition 4, the nth trial uniquely maps to a bit string of length $m=\lfloor\log_2 n\rfloor$. Thus, the expected bit length is given approximately by

$$\mathbb{E}[\log_2\mathrm{Q}]=\log_2\left(\frac{\left(\frac{m}{r}\right)^m\left(\frac{m}{r}-m\right)!}{\left(\frac{m}{r}\right)!}\right) \tag{e}$$

$$=m\log_2\left(\frac{m}{r}\right)+\log_2\left(\frac{m}{r}-m\right)!-\log_2\left(\frac{m}{r}\right)!\text{ bits}\,. \tag{f}$$

By Stirling's approximation,

$$\log_2 n!\approx n\left(\log_2 n-\log_2\mathrm{e}\right)\,. \tag{g}$$

and so the expectation may be rewritten approximately as

$$\begin{aligned}\mathbb{E}[\mathrm{Q}]\approx m\log_2\left(\frac{m}{r}\right)-\frac{m}{r}\left(\log_2\left(\frac{m}{r}\right)-\log_2\mathrm{e}\right)+\\ \left(\frac{m}{r}-m\right)\left(\log_2\left(\frac{m}{r}-m\right)-\log_2\mathrm{e}\right)\text{ bits}\,.\end{aligned} \tag{h}$$

Since we are interested in the expected *bits per element*, we divide the expectation by m and after further simplification arrive at

$$\begin{aligned}\log_2\left(\frac{m}{r}\right)-\frac{1}{r}\log_2\left(\frac{m}{r}\right)+\\ \left(\frac{1}{r}-1\right)\log_2\left(\frac{m}{r}-m\right)+\log_2\mathrm{e}\text{ bits/element}\,.\end{aligned} \tag{i}$$

After further simplification, we arrive at

$$\log_2\mathrm{e}-\left(\frac{1}{r}-1\right)\log_2\left(\frac{1}{1-r}\right)\text{bits/element}\,. \tag{j}$$

□

The entropy of Q is given by

$$\frac{-(1-p)}{p}\log_2(1-p) - \log_2(p) \tag{15}$$

$$\left(\frac{1}{p} - 1\right)\log_2\left(\frac{1}{1-p}\right) + \log_2\frac{1}{p} \tag{16}$$

Note that according to **??**, algorithm 1 has an exponential time complexity with respect to the cardinality of the input set $\mathbb{S}$. As a result, algorithm 1 is not a practical algorithm for any reasonably large m. However, it is intended to illustrate theoretical properties useful to data structures that implement *oblivious* sets and maps[4, 5] based on the perfect hash function. Simple and efficient algorithms exist [1, 2].

The theoretical lower-bound of a *minimal* perfect hash function is given by the following postulate.

Postulate 1. *The theoretical lower-bound for* minimal *perfect hash functions has an expected coding size given approximately by*

$$1.44\,\text{bits/element}\,. \tag{17}$$

Theorem 4. *The cryptographic perfect hash generator given by algorithm 1 results in a* minimal *perfect hash function that obtains the theoretical lower-bound of* 1.44 bits/element *by invoking* `make_perfect_hash(`$\cdot$, $r = 1$`)`.

Proof. By eq. (14), letting $r \to 1^+$ for the *minimal* perfect hash results in an expected coding size given by

$$\log_2 \mathrm{e} - \lim_{a\to 0^-} a\log_2\frac{1}{a} \approx 1.44\text{ bits/element}\,, \tag{a}$$

which is the expected lower-bound given by postulate 1. □

This is as expected, since algorithm 1 finds the *smallest* perfect hash function that is a function of a random oracle for any load factor $0 < r \leq 1$ in which the distribution of the sets are uniformly distributed. The following corollary follows as a result.

Corollary 1. *The lower-bound for perfect hash functions with a load factor* $0 < r \leq 1$ *is given by eq.* (14).

As r goes to 0, the expected bits per element goes to 0. t Note that the *probability mass function* of the random bit string found for b is known exactly. Thus, if the desire is to *serialize* the perfect hash function for transmission or storage, shorter bit strings may be assigned to more probable bit strings. This representation is not usable *in-place*, i.e., the serialization must be decoded, but it can reduce transmission or storage cost.

2.1 Entropy

The entropy of the random bit length N is given by

$$? \tag{18}$$

Proof.

$$-\sum_{n=0}^{\infty} \log_2\big(p_{\mathrm{N}}(n \,|\, m, r)\big)\, p_{\mathrm{N}}(n \,|\, m, r)\,. \tag{a}$$

□

Appendices

A k-perfect hash function

A k-perfect hash function permits up to k collisions and is therefore a *generalization* of the hashing family, i.e., a *perfect hash function* is a special case where $k = 0$.

Algorithm 2: k-perfect hash function generator

in : $\mathbb{S}$ is a set, r is a load factor, and k is the maximum number of collisions allowed.
out : A ℓ-perfect hash function of $\mathbb{S}$ with a load factor r and $\ell \leq k$.

```
function make_perfect_hash(𝕊, r, k)
    m ← |𝕊|;
    N ← ⌈m/r⌉;
    for n ← 1 to ∞ do
        // We give a name to the hash function that is coded by n′. Note that
           ++ is the concatenation operator.
        ĥ(x) ≡ h(x++n′) mod N;
        ℓ ← 0;
        for x ∈ 𝕊 do
            h ← ĥ(x);
        end
        if ℓ ≤ k then
            // This tuple is sufficient to code the perfect hash function for 𝕊
               that has the minimum bit length.
            return (n′, N);
        end
    end
```

The probability that t collisions occur for a particular bit string in algorithm 2 is given by the following theorem.

Theorem 5. *The probability that a bit string $b \in \mathbb{B}$ results in a t-perfect hash function $\mathtt{h}_{\mathbb{S}}$ in algorithm 2 is given by*

$$p(m, N) = \frac{N!}{N^m (N-m)!}\,, \tag{19}$$

where $m = |\mathbb{S}|$ and N is the maximum hash of $\mathtt{h}_{\mathbb{S}}$.

Proof. Suppose we have a set $\mathbb{S} = \{x_1, \ldots, x_m\}$, $t \geq m$. The probability that $x_1, \ldots, x_t$ collide and

the rest $x_{t+1}, \ldots, x_m$ do not collide is given by

$$\frac{1}{N^t} \prod_{i=1}^{m-t} \frac{N-i}{N} = \frac{(N-1)!}{N^m (N-m+t-1)!} \,. \tag{a}$$

We are actually interested in *any* t elements $\{x_{j_1}, \ldots, x_{j_t}\}$ colliding, not just $x_1, \ldots, x_t$. There are $\binom{m}{t}$ ways to choose t of elements in $\mathbb{S}$ as a collision. Thus, the probability that ...

$$p(m,t,N) = \frac{m!}{t!(m-t)!} \frac{(N-1)!}{N^m (N-m+t-1)!} \,. \tag{b}$$

We are interested in the first case when t collisions occur, which is a geometric distribution with probability of success $p(m,t,N)$ as given by the discrete random variable

$$\mathrm{Q} \sim \mathrm{GEO}\big(p(m,t,N)\big) \,. \tag{c}$$

The expected number of trials for the geometric distribution is given by

$$\mathbb{E}\left[\mathrm{Q}\right] = \frac{1}{p(m,t,N)} = \frac{t!(m-t)!N^m(N-m+t-1)!}{m!(N-1)!} \,. \tag{d}$$

TODO: Taylor approximation. Should work out well...but maybe not worth doing...i.e., this is close enough?

By definition 4, the nth trial uniquely maps to a bit string of length $m = \lfloor \log_2 n \rfloor$. Thus, the expected bit length is given approximately by

$$\mathbb{E}[\log_2 \mathrm{Q}] = \log_2 \left(\frac{t!(m-t)!N^m(N-m+t-1)!}{m!(N-1)!} \right) \tag{e}$$

$$= \log_2 t! + \log_2 (m-t)! + m \log_2 N + \log_2 (N-m+t-1)! - \log_2 m! - \log_2 (N-1)! \text{ bits} \,. \tag{f}$$

By Stirling's approximation,

$$\log_2 n! \approx n \left(\log_2 n - \log_2 \mathrm{e} \right) \,. \tag{g}$$

and so the expectation may be rewritten approximately as

$$\begin{aligned} \mathbb{E}[\mathrm{Q}] \approx\ & m \log_2 N + t \left(\log_2 t - \log_2 \mathrm{e} \right) + \\ & (m-t) \big(\log_2 (m-t) - \log_2 \mathrm{e} \big) + \\ & (N-m+t-1) \big(\log_2 (N-m+t-1) - \log_2 \mathrm{e} \big) - \\ & m \left(\log_2 m - \log_2 \mathrm{e} \right) - \\ & (N-1) \big(\log_2 (N-1) - \log_2 \mathrm{e} \big) \text{ bits} \,. \end{aligned} \tag{h}$$

$$\begin{aligned} \mathbb{E}[\mathrm{Q}] \approx\ & m \log_2 N + t \log_2 t + (m-t) \log_2 (m-t) + \\ & (N-m+t-1) \log_2 (N-m+t-1) - \\ & m \log_2 m - (N-1) \log_2 (N-1) - (t-m) \log_2 \mathrm{e} \text{ bits} \,. \end{aligned} \tag{i}$$

Dividing by m results in

$$\begin{aligned} \mathbb{E}[\mathrm{Q}] \approx\ & \log_2 N + \frac{t}{m} \log_2 t + \left(1 - \frac{t}{m} \right) \log_2 (m-t) + \\ & (\frac{N}{m} - 1 + \frac{t}{m} - \frac{1}{m}) \log_2 (N-m+t-1) - \\ & \log_2 m - \frac{N-1}{m} \log_2 (N-1) - \left(\frac{t}{m} - 1 \right) \log_2 \mathrm{e} \text{ bits} \,. \end{aligned} \tag{j}$$

Letting $\eta = \frac{t}{m}$ yields

$$\begin{aligned}\mathbb{E}[\mathrm{Q}] \approx\ & \log_2 N + \eta \log_2 m\eta + (1-\eta)\log_2(m(1-\eta)) + \\ & (\frac{N}{m} - 1 + \eta - \frac{1}{m})\log_2(N - m(1+\eta) - 1) - \\ & \log_2 m - \frac{N-1}{m}\log_2(N-1) - (\eta - 1)\log_2 \mathrm{e}\ \text{bits}\,.\end{aligned} \tag{k}$$

Letting $r = \frac{m}{N}$ yields

$$\begin{aligned}\mathbb{E}[\mathrm{Q}] \approx\ & \log_2\left(\frac{m}{r}\right) + \eta \log_2 m\eta + (1-\eta)\log_2(m(1-\eta)) + \\ & \left(\frac{1}{r} - 1 + \eta - \frac{1}{m}\right)\log_2\left(\frac{m}{r} - m(1+\eta) - 1\right) - \\ & \log_2 m - \frac{\frac{m}{r} - 1}{m}\log_2\left(\frac{m}{r} - 1\right) - (\eta - 1)\log_2 \mathrm{e}\ \text{bits}\,.\end{aligned} \tag{l}$$

Simplifying more yields

$$\begin{aligned}\mathbb{E}[\mathrm{Q}] \approx\ & -\log_2 r + \eta\log_2\eta + \log_2 m + \log_2(1-\eta) - \eta\log_2(1-\eta) + \\ & \left(\frac{1}{r} - 1 + \eta - \frac{1}{m}\right)\log_2\left(m\left(\frac{1}{r} - 1 - \eta\right) - 1\right) - \\ & \left(\frac{1}{r} - \frac{1}{m}\right)\log_2\left(\frac{m}{r} - 1\right) + (1-\eta)\log_2 \mathrm{e}\ \text{bits}\,.\end{aligned} \tag{m}$$

Simplifying more yields

$$\begin{aligned}\mathbb{E}[\mathrm{Q}] \approx\ & -\log_2 r + \eta\log_2\eta + \log_2 m + \log_2(1-\eta) - \eta\log_2(1-\eta) + \\ & \left(\frac{1}{r} - 1 + \eta - \frac{1}{m}\right)\log_2\left(m\left(\frac{m - mr - \eta mr - r}{mr}\right)\right) - \\ & \left(\frac{1}{r} - \frac{1}{m}\right)\log_2\left(\frac{m}{r} - 1\right) + (1-\eta)\log_2 \mathrm{e}\ \text{bits}\,.\end{aligned} \tag{n}$$

Here:

$$\begin{aligned}\mathbb{E}[\mathrm{Q}] \approx & \log_2 \mathrm{e} - \eta\log_2(1-\eta) + \eta\log_2\eta - \\ & \eta\log_2 \mathrm{e} + \log_2(1-\eta) + \eta\log_2(\eta m - 1) - \\ & \frac{\log_2(\eta m - 1)}{m} + \frac{\log_2(m-1)}{m} + \log_2\left(\frac{m}{m-1}\right)\end{aligned} \tag{o}$$

□

B Probability mass of random bit length

The bit length of the perfect hash function found by algorithm 1 has an uncertain value with respect to (random) sets and therefore we may model it as a random variable as given by the following definition.

Algorithm 3: Bit length sampler of the cryptographic perfect hash function

in : The number m is the cardinality of the perfect hash set and the number r is the load factor.

out : The *minimum bit length* n conditioned on a random set of cardinality m and a load factor r.

```
function sample_bit_length(m, r)
    for i ← 1 to m do
        x ← randomly draw a bit string from 𝔹 without replacement;
        𝕊 ← 𝕊 ∪ {x};
    end
    (b_n, N) ← make_perfect_hash(𝕊, r);
    // The integer N must also be coded, which we assume has a constant bit
       length.
    return n + O(1);
```

Definition 10. *The perfect hash function generated by algorithm 1 has a random bit length given by*

$$\mathrm{N} = \texttt{sample_bit_length}(m, r) \tag{20}$$

where m is the cardinality of the random set and r is the load factor.

Note that if the distribution of sets is not uniformly distributed as given by `sample_bit_length`, then better algorithms than algorithm 1 are possible, i.e., the lower-bounds given by **??** are with respect to "uniformly distributed" random sets and smaller lower-bounds are possible if the random sets are non-uniformly distributed.

The probability mass of the random bit length N is given by the following theorem.

Theorem 6. *The random bit length* N *has a probability mass function given by*

$$p_{\mathrm{N}}(n \,|\, m, r) = q^{2^n - 1}\left(1 - q^{2^n}\right) . \tag{21}$$

where m is the cardinality of the random set, r is the load factor, and $q = 1 - p(m, r)$,

Proof. Each iteration of the loop in algorithm 1 has a collision test which is Bernoulli distributed with a probability of success $p(m, r)$, where success denotes no collision occurred. We are interested in the random length N of the bit string when this outcome occurs.

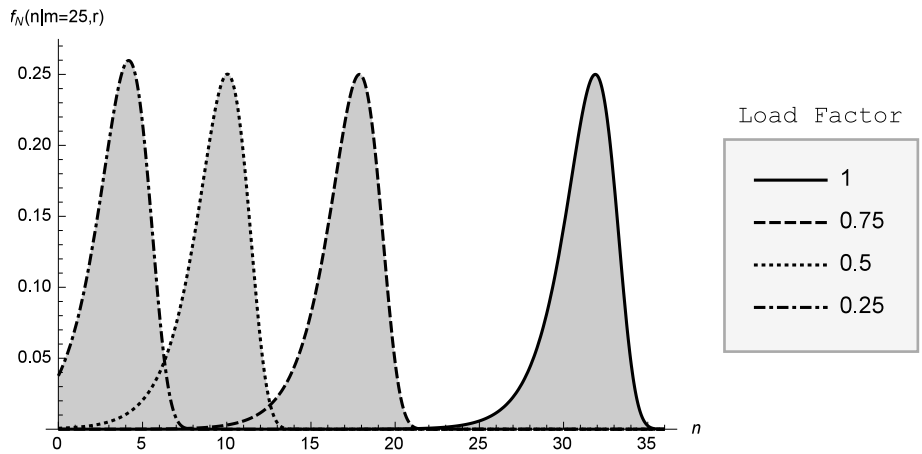
For the random variable N to realize a value n, every bit string smaller than length n must fail and a bit string of length n must succeed. There are $2^n - 1$ bit strings smaller than length n and each one fails with probability q, and so by the product rule the probability that they all fail is given by

$$q^{2^n - 1} . \tag{a}$$

Given that every bit string smaller than length n fails, what is the probability that every bit string of length n fails? There are 2^n bit strings of length n, each of which fails with probability q as before and thus by the product rule the probability that they all fail is q^{2^n}, whose complement, the probability that not all bit strings of length n fail, is given by

$$1 - q^{2^n} . \tag{b}$$

Figure 2: Probability mass function of random bit length N



By the product rule, the probability that every bit string smaller than length n fails and a bit string of length n succeeds is given by the product of (a) and (b),

$$q^{2^n-1}\left(1-q^{2^n}\right). \tag{c}$$

For eq. (c) to be a probability mass function, two conditions must be met. First, its range must be a subset of $[0,1]$. Second, the summation over its domain must be 1.

The first case is trivially shown by the observation that q is a positive number between 0 and 1 and therefore any non-negative power of q is positive number between 0 and 1.

The second case is shown by calculating the infinite series

$$S = \sum_{n=0}^{\infty} q^{2^n-1}\left(1-q^{2^n}\right) \tag{d}$$

$$= \sum_{n=0}^{\infty} q^{2^n-1} - q^{2^{n+1}-1}. \tag{e}$$

Explicitly evaluating this series for the first 4 terms reveals a telescoping sum given by

$$S = (1-q) + (q-q^3) + (q^3-q^7) + (q^7-q^{15}) + \cdots, \tag{f}$$

where everything cancels except 1. □

In fig. 2, we plot the probability mass function of N conditioned on $m = 25$ and several different load factors. We see that the probability mass is peaked and unimodal and tends to nearly zero everywhere except over a concentrated interval around its expected value.

The expected size was already computed, but the probability mass function contains everything there is to know about the distribution of N, not just the *expected* value. However, for illustration, we show how the expected value may be computed.

Theorem 7. *The expected bit length of* N *conditioned on random sets of cardinality* m *and a load factor* r *is given by*

$$\mathbb{E}[\mathrm{N}] = \sum_{j=1}^{\infty} q^{2^j-1}, \tag{22}$$

where $q = 1 - p(m, r)$.

Proof. The expectation of N is given by

$$\sum_{j=0}^{\infty} j q^{2^j - 1} \left(1 - q^{2^j}\right) \tag{a}$$

$$= \sum_{j=0}^{\infty} j \left(q^{2^j - 1} - q^{2^{j+1} - 1}\right) . \tag{b}$$

Explicitly evaluating this series for the first 4 terms reveals a converging sum given by

$$0 + (q - q^3) + 2(q^3 - q^7) + 3(q^7 - q^{15}) + \cdots \tag{c}$$

$$= q + q^3 + q^7 + q^{15} + \cdots . \tag{d}$$

□

When we numerically evaluate eq. (22) for various m and r, the results are in agreement with eq. (14).

Glossary

finite set A finite collection of distinct elements from a universe of elements. 1

hash function A hash function maps input of arbitrary length to output of fixed-length. 2

load factor The proportion of indexes between 1 and the maximum index which are mapped to by the perfect hash function. 3, 11

perfect hash function A hash function that guarantees no collisions among members of a specified set. 3, 13

set An unordered collection of distinct elements from a universe of elements. 1

List of Figures

1 Perfect hash function 3
2 Probability mass function of random bit length N 12

List of Algorithms

1 Perfect hash function generator 4
2 k-perfect hash function generator 8
3 Bit length sampler of the cryptographic perfect hash function 11

References

[1] Martin Dietzfelbinger Djamal Belazzougui, Fabiano C. Botelho. Compress, hash and displace. URL `http://cmph.sourceforge.net/papers/esa09.pdf`.

[2] Nivio Ziviani Fabiano C. Botelho1, Rasmus Pagh. Simple and space-efficient minimal perfect hash functions. URL `http://cmph.sourceforge.net/papers/wads07.pdf`.

[3] Alex Towell. The approximate map abstract data type. .

[4] Alex Towell. The perfect hash filter: A fast and efficient implementation of the static approximate set and the oblivious set abstract data types. .

[5] Alex Towell. The perfect map filter: A fast and efficient implementation of the approximate map abstract data type. .